

Logan Sutton
ID: 11798384
Cpts 360
PA1 Report

I implemented my cache simulator by first creating 3 different structures: cacheline, cacheset, and cache. The cacheline structure contains the tag, validity, and least recently used information. The cacheset structure contains an array of cachelines. The cache structure contains an array of cachesets, and all of the information corresponding to the cache, such as the set bits, number of sets, block bits, etc. I use 3 global variables, hits, misses, and evictions to conveniently increment as the cache is utilized. I also created a constructor function to make a cache using the parameters passed in from the command line (set bits, block bits, associativity, etc.). The constructor function allocates the appropriate amount of memory based on the input parameters and constructs the cache accordingly.

After reading the command line arguments, I create the cache using the constructor function. Then, I call the read_file() function, which opens and reads the provided file. After parsing through a line of input data, the access_cache() function is called. Note that I convert the hex address to decimal format (unsigned long long), and pass that into the access_cache() function.

The access_cache() function follows the following procedure:

1. Extract the tag by using bitwise operations on the address.
2. Extract the set index by using bitwise operations on the address.
3. Loop through each cache line at the corresponding set index checking for matching tag and validity. If these two conditions are met, we have a hit and go to step 8. Otherwise, we have a miss.
4. If we have a miss, I then check for an empty line to insert into. During this process, I also keep track of the least recently used index, in the case that we need to evict the least recently used cacheline. If an empty line is found, its index is stored.
5. If no empty line is found, I set the index to insert at as the least recently used index.
6. After finding a place to insert into, I increment the least recently used attributes of each cache line in the set.
7. I then insert at the correct spot by updating the tag to the new tag, the least recently used value to 1, and the validity to 1.
8. Lastly, I check the operation to see if it is a modify operation. If it is a modify, then I increment the hit count by 1, as modify operations will be a hit after the first access.

Note: throughout the entire procedure, if the v flag was set, then the hits, misses, and evictions are printed in the correct format.

Before exiting the program and after the results are displayed, I free the cache's memory.